Solution Architecture

Date	13 July 2026
Team ID	xxxxxx
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Solution Architecture Diagram:

Presentation / Client Layer

[e.g., Web App (React), Mobile App (Flutter), Admin Dashboard]

Web UI: Home/Dashboard, Applicant Input Form, Prediction Result, Prediction History (HTML/CSS + Jinja2, served by Flask), accessed via web browser by the User/Credit Analyst.

↓↑

API Gateway / Load Balancer

[e.g., Nginx, AWS API Gateway, Express Server]

No separate API gateway/load balancer - Flask's built-in URL routing (Route Handling) dispatches requests directly to the /predict endpoint.

↓↑

Auth Service	Core Logic Service	External API Integrations
[e.g., JWT, OAuth2] Not implemented in the current version - no login/authentication layer.	*[e.g., Data Processing, Machine Learning Models]* Flask application layer: form validation, data pre-processing (scaling/encoding), loading the trained ML model (joblib/pickle), generating the prediction, and returning the result. Machine Learning layer: trained classifiers (Logistic Regression, Random Forest, XGBoost, Decision Tree), model evaluation/comparison, and best-model selection.	*[e.g., Payment Gateway, Email Service]* IBM Watson Machine Learning service (hosts/serves the deployed model) and IBM Cloud Object Storage (stores model artifacts); GitHub used for source code storage as part of the deployment pipeline.

↓↑

Data / Storage Layer

[e.g., PostgreSQL Database, MongoDB, AWS S3]

Raw data sources (applicant details, credit history, payment history, demographics) -> data pre-processing & feature engineering -> 80/20 train/test split -> processed training dataset. Trained model file (model.pkl) and preprocessing objects (scaler.pkl / encoder.pkl) are stored locally and in IBM Cloud Object Storage after deployment.

Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Web-based UI where the credit analyst enters applicant details, submits the form, and views the prediction result and past prediction history.	HTML5, CSS3, Jinja2 (Flask templating)
API Gateway	Routes incoming HTTP requests from the UI to the appropriate Flask view function (e.g., the /predict route); no separate gateway/load balancer is used.	Flask routing (Werkzeug)
Microservices	Core application and ML logic: validates form input, pre-processes/encodes features, loads the trained model, generates the approval/rejection prediction, and returns it to the UI. Also covers model training, evaluation, and selection of the best-performing classifier.	Flask, scikit-learn, XGBoost, pandas, numpy, joblib/pickle
Database	Stores raw and processed applicant/credit-history data used for training, plus the serialized model and preprocessing objects (scaler/encoder) used at prediction time.	CSV files (application_record.csv, credit_record.csv), joblib/pickle model & preprocessing artifacts, IBM Cloud Object Storage
Deployment / MLOps	Code is developed and tested locally, pushed via Git to a GitHub repository, then deployed to IBM Cloud; the trained model is registered with IBM Watson Machine Learning and its artifacts stored in IBM Cloud Object Storage, exposing the app as a live public web application.	Git, GitHub, IBM Cloud, IBM Watson Machine Learning, IBM Cloud Object Storage

Detailed Architecture Diagram
Full technical architecture, showing the User, Presentation, Application, Machine Learning, Data, Model Storage, and Deployment layers.

SMARTBRIDGE
Let's Bridge the Gap

SKILL WALLET
A SmartBridge Product

Technical Architecture – Credit Card Approval Prediction Using IBM Watson Machine Learning